

CAS-HTC-report-EN

Classifying cyber-fraud reports: where to spend a limited annotation budget?

Bibien Limido — [bibien.limido@ncsc.ch]

Federal Office for Cyber Security (NCSC)

CAS Advanced Machine Learning, University of Bern — June 2026

Note: the incident examples used here were pseudonymised by hand.

Abstract

The NCSC receives about 60,000 cyber-fraud reports per year, in four languages. They must be sorted into an internal, hierarchical taxonomy that keeps changing. Building a training set from these reports is expensive: many have no usable text, and both picking and checking examples is done by hand. So we work in a few-shot setting.

The project question is simple: **when data is scarce and expensive, where should the effort go — describing the categories or labelling examples — and which method gets the most out of it?**

We compare two families of methods over four knowledge settings of growing cost: the category names (R0a, free); one expert description per leaf (R0b, about 1 h for all 13); 3 labelled examples per leaf (R3, about 2 h); 8 examples per leaf (R8, about 5 h). The first family is a small multilingual sentence encoder (`paraphrase-multilingual-mpnet-base-v2`) used in zero-shot similarity, linear probe, and contrastive fine-tuning. The second is a local LLM (`gemma4:26b` , `gemma4:e4b`) used with prompting. All systems share the same splits over 5 seeds; the comparison is paired by seed.

Main results: (1) **descriptions give the best return** (gain per hour spent), in both families: +0.286 leaf accuracy on the encoder side, +0.107 on the LLM side. (2) In-context examples **hurt** the LLMs compared with descriptions alone (0/5 seeds positive). (3) At the R8 budget, the 26B LLM still leads contrastive fine-tuning on accuracy (0.911 vs 0.795), but it pays a long prompt on every call. (4) The encoder produces **probabilities** natively, which are useful for triage: at a 5% tolerated error, the linear probe automates 48% of cases, contrastive fine-tuning 58%. Results are established on one branch of the real taxonomy (13 leaves, 234 English examples).

Table of contents

- [Classifying cyber-fraud reports: where to spend a limited annotation budget?](#)
 - [Abstract](#)
 - [Table of contents](#)
 - [1. Introduction](#)
 - [2. Data](#)
 - [2.1 Corpus and taxonomy](#)
 - [2.2 Category descriptions \(R0b\)](#)
 - [2.3 Incident examples \(R3 and R8\)](#)
 - [2.4 Label quality control](#)
 - [3. Exploratory analysis](#)
 - [3.1 Lengths](#)
 - [3.2 The encoder window](#)
 - [3.3 The LLM window](#)
 - [4. Methods](#)
 - [4.1 Encoder family](#)
 - [4.2 LLM family](#)
 - [4.3 Protocol](#)
 - [4.4 Metrics](#)
 - [4.5 Reproducibility](#)
 - [5. Results](#)
 - [5.1 The full grid](#)
 - [5.2 Encoder-family errors](#)
 - [5.3 LLM-family errors](#)
 - [5.4 Confidence analysis](#)
 - [5.5 Effect of contrastive fine-tuning on the latent space](#)
 - [6. Discussion](#)
 - [6.1 Significance and uncertainty](#)
 - [6.2 Economic reading](#)
 - [6.3 Limitations](#)
 - [7. Conclusion and outlook](#)
 - [Acknowledgements](#)
 - [References](#)
 - [More data](#)
-

1. Introduction

The NCSC receives cyber-fraud reports (phishing, scams, malware) written in free text by the public, in German, French, English, and Italian. Sorting them into a hierarchical taxonomy feeds triage, statistics, and public-awareness work. This project lays the ground for small-scale automation.

With about 60,000 reports per year, you might think the data is plentiful. It is not, for three reasons. **Double imbalance:** German dominates (>60%), Italian is rare (<3%), and the categories themselves are uneven; some crossings have fewer than 40 examples. **Cost of building the set:** many reports have no usable text; picking and checking cases is manual. **Ageing:** the taxonomy changes with the threats, creating fresh categories with zero examples. On top of that, data protection: examples must be pseudonymised, which is also a reason to run inference locally (§4).

The few-shot setting follows from these limits. The budget of **8 training examples per category** follows the scale used in SetFit-style few-shot experiments (Tunstall et al., 2022); with **10 test examples** per category, this budget is a hard constraint of the protocol.

Two families can consume these forms of knowledge, and the literature does not pick a winner up front. On classification tasks, a small fine-tuned encoder still often beats LLM prompting (Edwards & Camacho-Collados, 2024; Bucher & Martini, 2024). But that work relies on *full* fine-tuning of the encoder, which is unstable in few-shot (Mosbach et al., 2021; Zhang et al., 2021). So we use variants that are robust in this setting: a pre-trained sentence encoder (Reimers & Gurevych, 2019), used in zero-shot similarity, in a linear probe (Alain & Bengio, 2017), and in contrastive fine-tuning inspired by SetFit. On the LLM side, knowledge goes through the prompt (Brown et al., 2020). Because the dataset is very small, the classifier predicts the leaf directly (flat design), and the hierarchy is brought back at evaluation time. This choice is consistent with the HTC literature, where hierarchy-aware modelling is not automatically better than strong flat baselines and where the choice of metrics strongly affects the conclusions (Silla & Freitas, 2011; Plaud et al., 2024).

Setting (budget)	Encoder family	LLM family
R0a (names, ~0)	similarity to names	zero-shot
R0b (descriptions, ~1 h)	similarity names + descriptions	zero-shot + descriptions
R3 (39 labels, ≈ 2 h)	linear probe (3) / SetFit (3)	in-context (3)
R8 (104 labels, ≈ 5 h)	linear probe (8) / SetFit (8)	in-context (8)

Table 1. Experimental grid. The settings are nested ($R0a \subset R0b$, $R3 \subset R8$). Contrastive fine-tuning needs pairs of examples, so it only exists at R3 and R8.

2. Data

2.1 Corpus and taxonomy

The corpus covers one branch of the real taxonomy: **234 English examples, 13 leaves** over 3 levels (20 nodes; 5 leaves at depth 2, 8 at depth 3). That is 13 leaves × 18 examples (8 for training, 10 for testing).

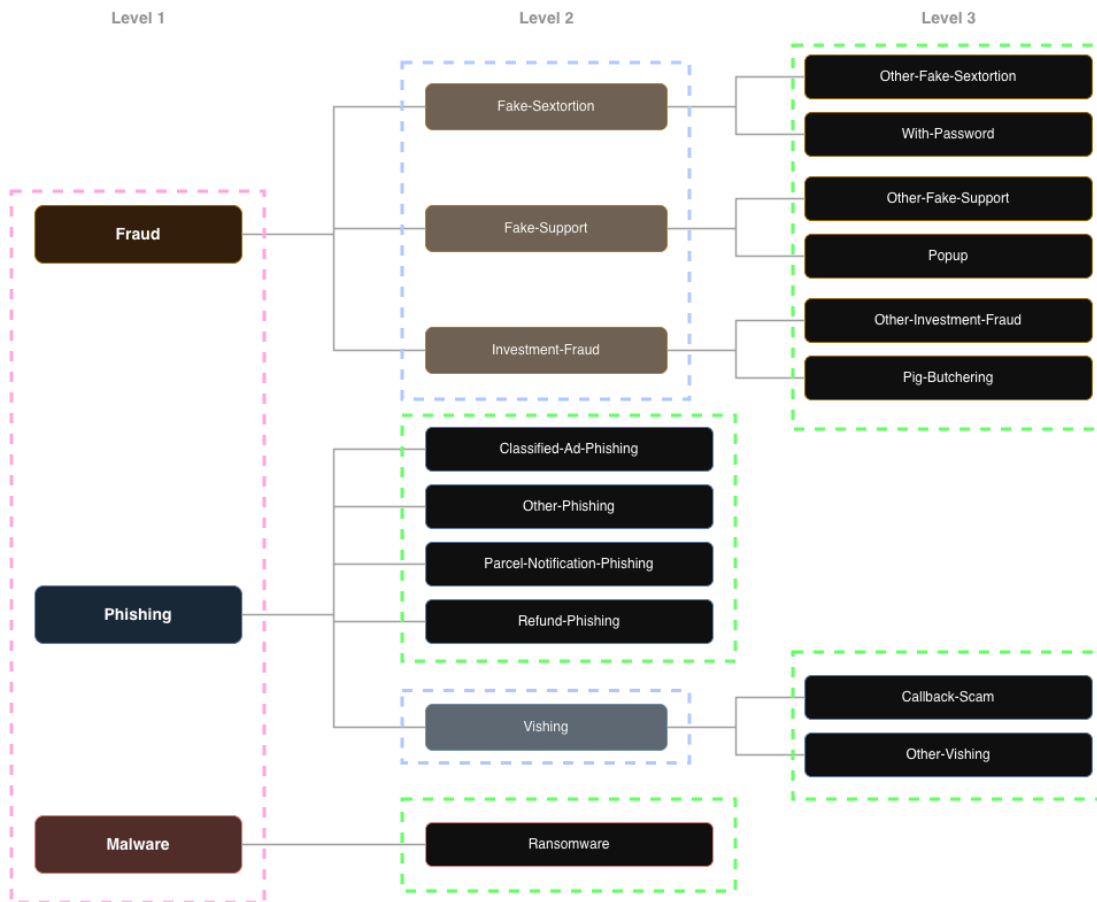


Figure 1. Branch of the incident taxonomy covered by the study.

The category knowledge defines the four settings. **R0a**: the names, free. **R0b**: one structured description per leaf (a `general` field and a list of `indicators`), added to the names — about 1 h of writing for all 13. **R3 / R8**: 3 then 8 labelled examples per leaf (39 then 104 labels, about 2 h then about 5 h).

2.2 Category descriptions (R0b)

The descriptions were written by the author from his knowledge of the taxonomy. They are not copied from the examples in the dataset. Each leaf has a typical scenario and concrete indicators. At use time, descriptions and names are joined together (cumulative setting). Example of the format:

```

"Malware/Ransomware": {
  "general": "Malicious software encrypting victim's files or systems and
demanding cryptocurrency ransom...",
  "indicators": [
    "Files renamed with unusual extensions (.lockbit, .conti, .akira)",
    "Ransom notes (README.txt) in folders",
    "Bitcoin/Monero ransom demands with deadlines",
    "Ransomware family names: LockBit, BlackCat, Conti, Akira"
  ]
}

```

These descriptions often encode what separates two sibling leaves: who starts the call (Callback-Scam vs Other-Vishing), whether or not a password is present (With-Password vs Other-Fake-Sextortion), a prior romantic relationship (Pig-Butchering vs Other-Investment-Fraud). This point matters for reading the results (§5.3, §6.3).

2.3 Incident examples (R3 and R8)

The examples are real cases, pseudonymised to remove the irrelevant PII (names, addresses, dates). The name of the **impersonated company** is kept: it is sometimes a telling clue. Close categories differ on fine details — the channel (popup, call, email), who starts the contact, whether there is a prior relationship — exactly the details encoded in the descriptions.

2.4 Label quality control

Before the experiments, three detectors looked for label errors in the embedding space: **k-NN disagreement** (the share of neighbours that share the leaf; a signal if < 0.3), **cross-validation self-confidence** (a confident but wrong prediction, in the style of confident learning (Northcutt et al., 2021), and **short texts** (< 5 words). These detectors flag examples that are hard *for this approach*, not necessarily errors; relabelling on this signal alone would have biased the comparison. The 34 flagged examples were reviewed by hand: 34 kept, 0 relabelled, 0 removed.

Dataset quality pre-screening — 34/234 examples flagged

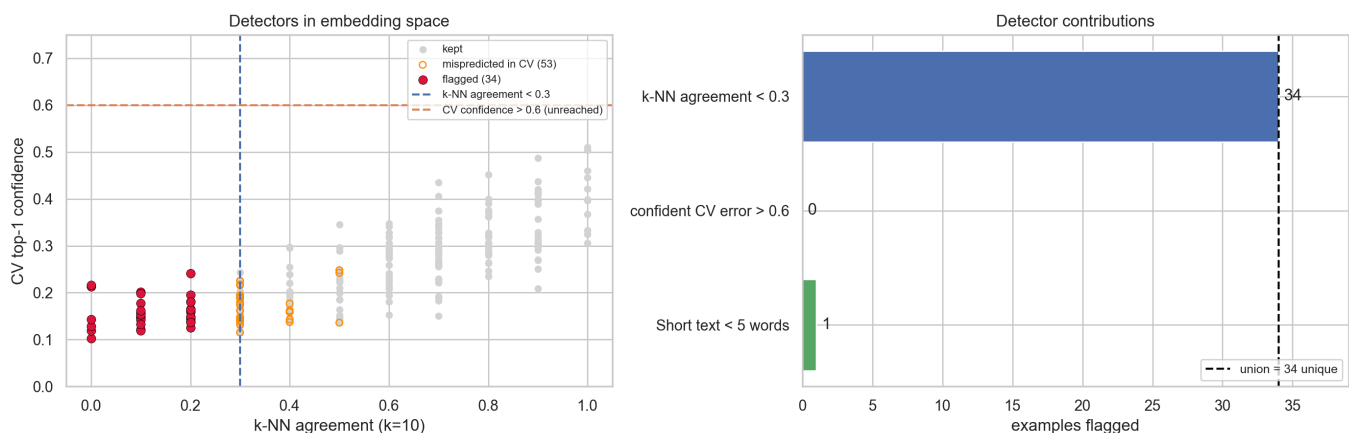


Figure 2. Examples flagged by the three label-quality detectors.

3. Exploratory analysis

3.1 Lengths

Median length: 72 words [P25 = 50, P75 = 131], maximum 619. In encoder tokens: median 109, P75 = 193, max 2,121 (median ratio 1.42 tokens/word).

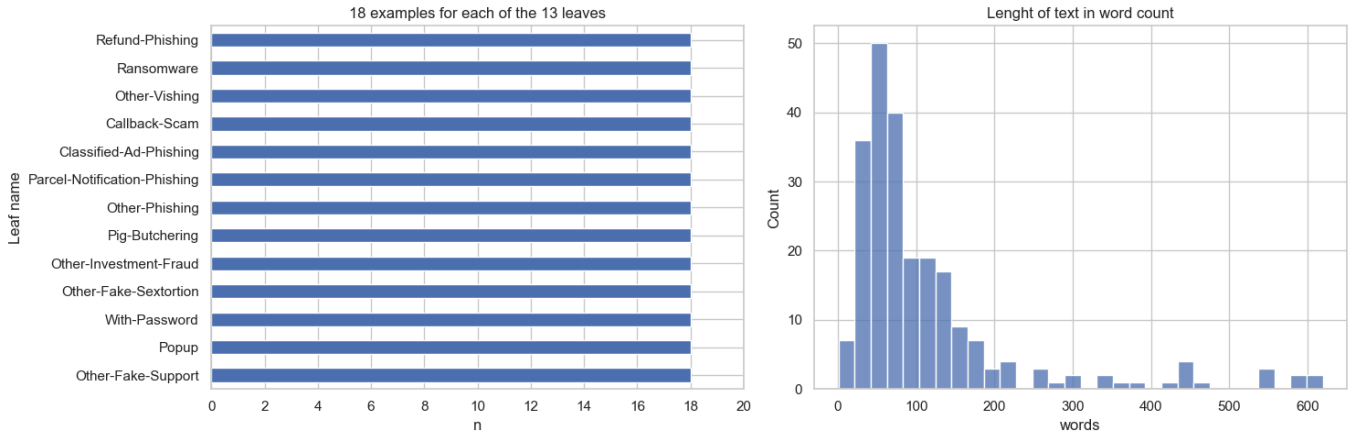


Figure 3. Left: 18 examples per leaf (balanced set); right: distribution of lengths in words.

3.2 The encoder window

The encoder cuts the input at 128 tokens (about 90 words). So some long texts are not seen in full. Does this lose useful information? Two arguments suggest not: the telling information is often at the start of the text, and the second part of a report often repeats the fraud email, which is not very informative. We checked this by measuring the linear-probe leaf accuracy as a function of the window (5 seeds):

8 tokens : 0.297 ± 0.022	64 tokens : 0.726 ± 0.014
16 tokens : 0.462 ± 0.023	128 tokens : 0.737 ± 0.017
32 tokens : 0.657 ± 0.033	256 tokens : 0.714 ± 0.015

Performance saturates between 64 and 128 tokens; going to 256 does not improve it ($\Delta = -0.023$). This matches the lead bias (Kedzie et al., 2018) and robustness to truncation (Sun et al., 2019). We keep 128 tokens.

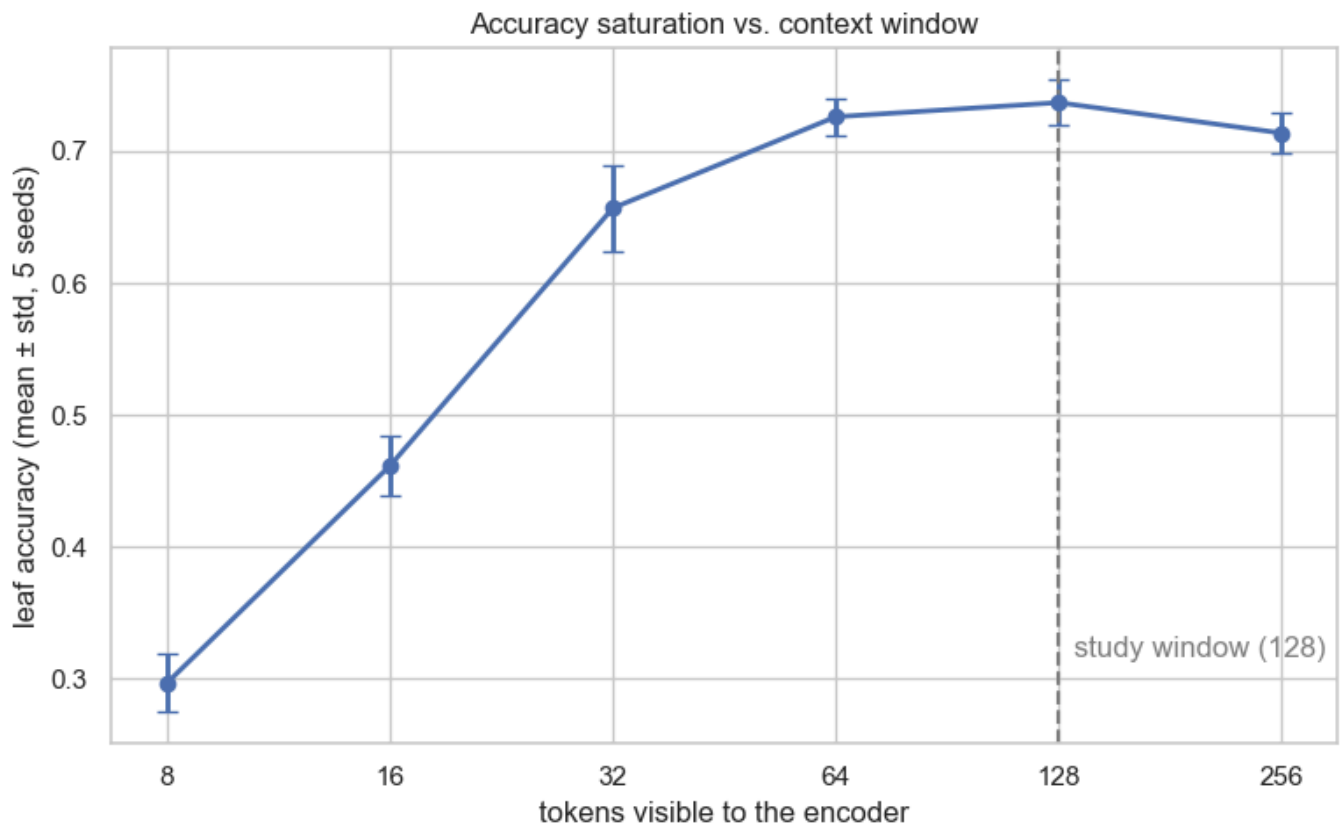


Figure 4. Linear-probe leaf accuracy by window size (5 seeds).

3.3 The LLM window

The LLM case is different: the same window must hold the whole prompt (taxonomy, descriptions, or examples) *and* the text to classify. The prompt grows fast with the setting. Measured with the real `gemma4` tokenizer (5 seeds $\times$ 130 texts), median sizes go from about 1,700 tokens in R0a/R0b to 7,000–10,500 in R3 and 19,000–25,900 in R8; the heaviest reaches about 28,500 tokens. At the standard window of 16,384 tokens, the heaviest R8 prompts were **silently truncated**. So we raised the window to 32,768 for R8, and an anti-truncation check verifies, before each batch, that the heaviest prompt is read in full.



Figure 5. Structure of the "flat" prompt given to the LLM.

4. Methods

4.1 Encoder family

Encoder: sentence-transformers/paraphrase-multilingual-mpnet-base-v2 (XLM-RoBERTa backbone, 278M parameters, 128-token window; see the Hugging Face model card). The multilingual choice is on purpose (the final goal is four languages), even though the corpus is in English.

- **R0a/R0b — zero-shot similarity.** The names (R0a) or descriptions + names (R0b) are encoded; the prediction is the nearest cosine neighbour. This is "dataless classification" (Chang et al., 2008; Yin et al., 2019); our contribution is to measure its cost and its return.
- **R3/R8 — linear probe.** Frozen, normalised embeddings + multinomial logistic regression (same settings everywhere).
- **R3/R8 — SetFit.** Contrastive fine-tuning of the encoder, then a logistic head (positive/negative pairs, CosineSimilarityLoss, 1 epoch). The `setfit` library is not compatible with our version of transformers, so the algorithm is re-implemented with sentence-transformers; the pairs, the loss, and the head stay those of SetFit (Tunstall et al., 2022).

4.2 LLM family

Models served locally with Ollama 0.30.8 on a Mac Studio (Apple Silicon M3 Ultra):

`gemma4:26b` and `gemma4:e4b`. The `26b` is a *mixture-of-experts* (about 26 billion parameters in total, about 4 billion active per token; see the Gemma 4 model card from Google and the Ollama page); so the contrast with `e4b` (about 4 billion "effective") is about total capacity and memory, not about active compute. Temperature 0, fixed seed. The text output is mapped to a valid leaf by a parser (exact match, substring, then fallback); over the whole grid, the global fallback was never triggered (0%). In total: 2 models × 4 conditions × 130 tests × 5 seeds = 5,200 calls, through a resumable loop (no duplicates).

4.3 Protocol

For each seed $s \in \{0, \dots, 4\}$: a balanced split of 8 train / 10 test per leaf (104 / 130 examples). Three properties make the comparison fair, and all comparisons in the report rely on them:

1. **Same splits across systems** at a fixed seed → a paired comparison, seed by seed.
2. **Test sets unchanged across settings:** only the training set changes; the test stays the same 130 examples, always separate from the training set.
3. **Nested budgets:** the 3 examples of R3 are a fixed subset of the 8 of R8. If adding examples lowers performance, that is a result, not a sampling artefact.

4.4 Metrics

Four questions, four families of metrics.

- **Leaf accuracy** and **leaf macro-F1** — *the right leaf?* The exact decision at the leaf level. A gap between the two shows errors concentrated on a few leaves.
- **hF1 micro** and **hF1 macro per node** — *is the error serious?* We unfold the predicted leaf into a full path and compare it with the true path (Silla & Freitas, 2011; Kiritchenko et al.,

2006; Kosmopoulos et al., 2015). Confusing two investment frauds is almost right; confusing a fraud with ransomware is not.

- **hF1-AUC** — *can we trust its confidence?* Building on the hierarchical-F1 metrics from the HTC literature (Plaud et al., 2024), we define hF1-AUC as the area obtained by sweeping every confidence threshold, which sums up the sorting (safe cases automated, doubtful cases sent to a human) in one number. It applies to the encoder; not to the LLM, which gives a label with no distribution.
- **Costs** — annotation budget and inference time per example.

Paired tests. At each seed, two systems give a pair of scores on the same examples. We compare the 5 paired differences. Two readings: the **sign test** (over 5 seeds, how many go the same way? 5/5 or 0/5 is the clearest verdict; 3/5 concludes nothing) and the **exact signed Wilcoxon**. At $n = 5$, the smallest p you can reach is 0.031 (one-sided): so a p of 0.03 is not a weak effect, it is the floor. Our conclusions are about the **sign** of the effects, not their size — 0.03 of accuracy is only about 4 examples out of 130.

4.5 Reproducibility

Five notebooks on a shared module (`helpers.py`): loading and EDA (`00`), similarity and linear probe (`01`), contrastive fine-tuning (`02`), LLM calls (`03`), recomputing the metrics (`04`). Embeddings are cached, the environment is frozen, and each artefact carries its metadata (encoder, commit, window), checked before any comparison. The artefacts also include two Module 6 notebooks that some choices rely on, as noted in the report.

Code and notebooks: https://github.com/elfreyer/CAS_HTC.

5. Results

5.1 The full grid

Setting	System	Annot. cost	Inf. cost	Leaf acc.	Macro-F1	hF1 micro	hF1 macro	hF1-AUC
R0a	LLM 26b	~0	0.59 s/ex	0.815 ± 0.024	0.814 ± 0.023	0.873 ± 0.018	0.846 ± 0.019	n/a
R0a	LLM e4b	~0	0.55 s/ex	0.463 ± 0.020	0.417 ± 0.030	0.627 ± 0.022	0.515 ± 0.028	n/a
R0a	Sim. names	~0	~5 ms/ex	0.294 ± 0.029	0.229 ± 0.034	0.452 ± 0.026	0.319 ± 0.030	0.572 ± 0.014

Setting	System	Annot. cost	Inf. cost	Leaf acc.	Macro-F1	hF1 micro	hF1 macro	hF1-AUC
R0b	LLM 26b	~1 h	0.60 s/ex	0.922 ± 0.016	0.920 ± 0.016	0.934 ± 0.011	0.928 ± 0.013	n/a
R0b	LLM e4b	~1 h	0.57 s/ex	0.822 ± 0.025	0.825 ± 0.025	0.867 ± 0.019	0.848 ± 0.022	n/a
R0b	Sim. descr.	~1 h	~5 ms/ex	0.580 ± 0.015	0.552 ± 0.011	0.712 ± 0.018	0.630 ± 0.013	0.805 ± 0.012
R3	LLM 26b	39 ex ≈ 2 h	0.70 s/ex	0.874 ± 0.026	0.872 ± 0.028	0.905 ± 0.024	0.891 ± 0.026	n/a
R3	LLM e4b	39 ex ≈ 2 h	0.66 s/ex	0.703 ± 0.042	0.683 ± 0.043	0.786 ± 0.036	0.733 ± 0.040	n/a
R3	Linear probe	39 ex ≈ 2 h	~5 ms/ex	0.648 ± 0.048	0.633 ± 0.055	0.776 ± 0.044	0.703 ± 0.054	0.565 ± 0.011
R3	SetFit	39 ex ≈ 2 h	~5 ms/ex	0.685 ± 0.046	0.677 ± 0.048	0.801 ± 0.039	0.738 ± 0.045	0.663 ± 0.020
R8	LLM 26b	104 ex ≈ 5 h	0.93 s/ex	0.911 ± 0.019	0.908 ± 0.022	0.928 ± 0.026	0.919 ± 0.025	n/a
R8	LLM e4b	104 ex ≈ 5 h	0.87 s/ex	0.737 ± 0.040	0.718 ± 0.047	0.800 ± 0.040	0.757 ± 0.047	n/a
R8	Linear probe	104 ex ≈ 5 h	~5 ms/ex	0.737 ± 0.019	0.722 ± 0.022	0.840 ± 0.020	0.782 ± 0.022	0.671 ± 0.007
R8	SetFit	104 ex ≈ 5 h	~5 ms/ex	0.795 ± 0.036	0.795 ± 0.036	0.865 ± 0.016	0.831 ± 0.026	0.879 ± 0.018

Table 2. Full grid (mean ± standard deviation over 5 seeds). hF1-AUC: n/a for the LLM (a label with no distribution).

The grid reads along two axes, and the winner changes with the axis: raw **accuracy** (useful when a human checks afterwards) and the **ability to sort by confidence** (to automate the safe cases). The budget figure sums up both as a function of cost.

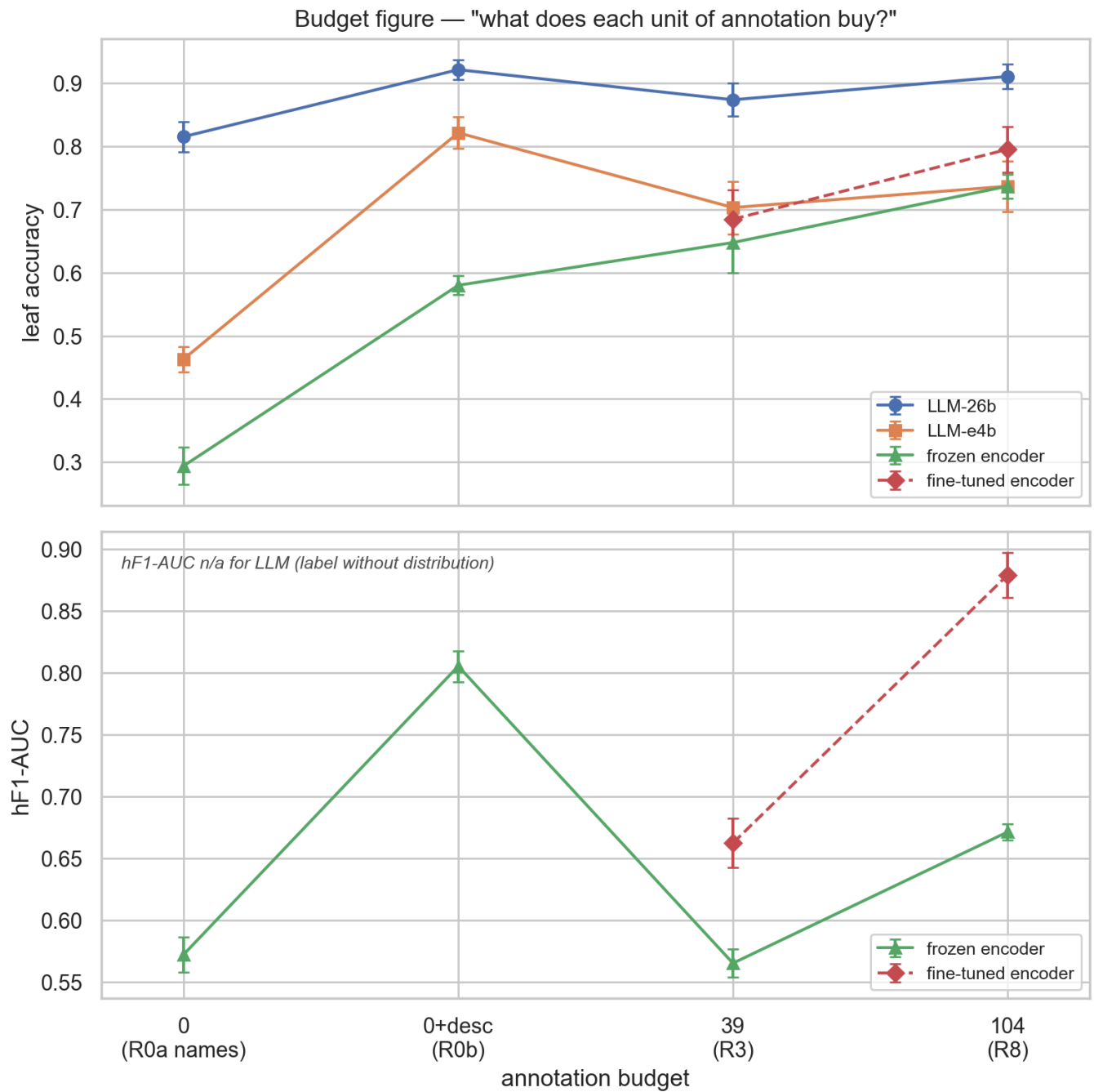


Figure 6. Return of each annotation unit. Top: leaf accuracy by budget (R0a → R8). Bottom: hF1-AUC by budget, encoder families only. The fine-tuned encoder appears only from R3 on. Means ± standard deviation over 5 seeds.

5.2 Encoder-family errors

At the R8 budget, the gap hF1 micro – leaf accuracy (0.103 for the linear probe, 0.070 for SetFit) says the main thing: when a system misses the leaf, it almost always keeps the right family. The confusion matrices (Figure 7) show this. With names only, many classes collapse onto a few attractor labels (mostly Parcel-Notification-Phishing). Descriptions remove most of this collapse and make the diagonal appear. The supervised R8 systems then concentrate almost all the remaining errors between sibling leaves.

Two pockets of confusion resist. Investment fraud (Pig-Butchering vs Other-Investment-Fraud), because both describe the same scam up to the romantic relationship. And the four flat Phishing leaves, where Other-Phishing, the catch-all, attracts the ambiguous cases. Only one confusion crosses families, and it makes business sense: Fake-Support and Vishing get a bit confused (two phone scams). The Malware case deserves a note: weak on frozen embeddings, Ransomware becomes one of the easiest leaves once trained (9–10/10), thanks to a distinctive vocabulary (ransom, .onion, ransomware names).

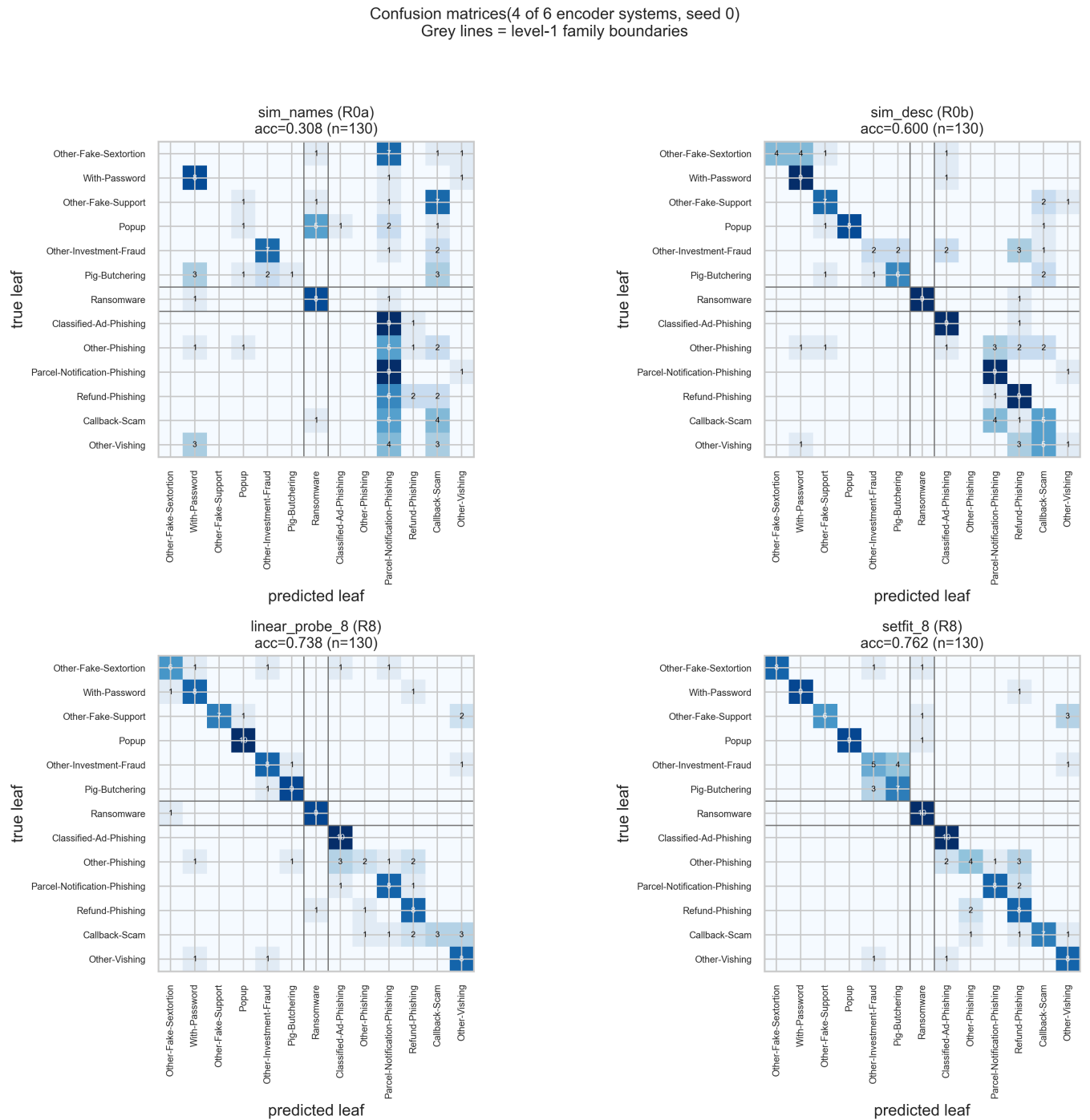


Figure 7. Confusion matrices (seed 0), from sim_names to SetFit R8. Grey lines = family borders; mass inside a block = confusion between siblings. The blocks fade as knowledge grows.

5.3 LLM-family errors

Without descriptions (R0a), the 26B mostly confuses one family with another: it drops Fake-Sextortion emails into Other-Phishing, for lack of a definition (26 errors out of 130, seed 0). Descriptions fix most of this: from R0a to R0b, accuracy goes from 0.815 to 0.922.

But be careful reading this number. By design, the descriptions give the model the decision borders between sibling leaves — the annotation rubric itself. So the 0.922 is not a "closed-book" zero-shot ability: it is the model boosted with this expert knowledge handed to it from the start, and it is not directly comparable with a setting without descriptions. A hint of how large this input is: fed the same descriptions, the `sim_desc` encoder tops out at 0.580 (Table 2).

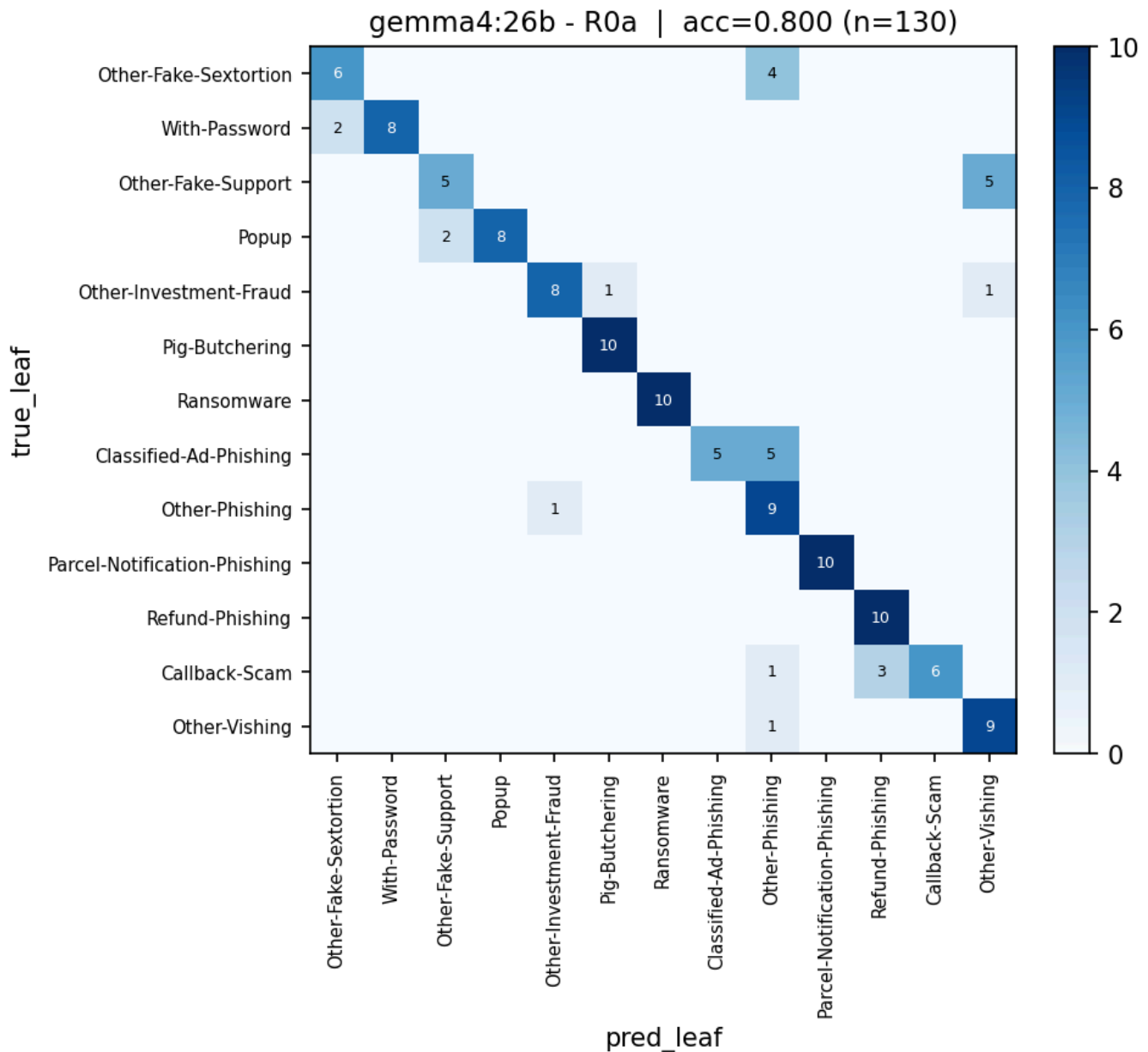


Figure 8. LLM 26B, R0a — confusion matrix (seed 0).

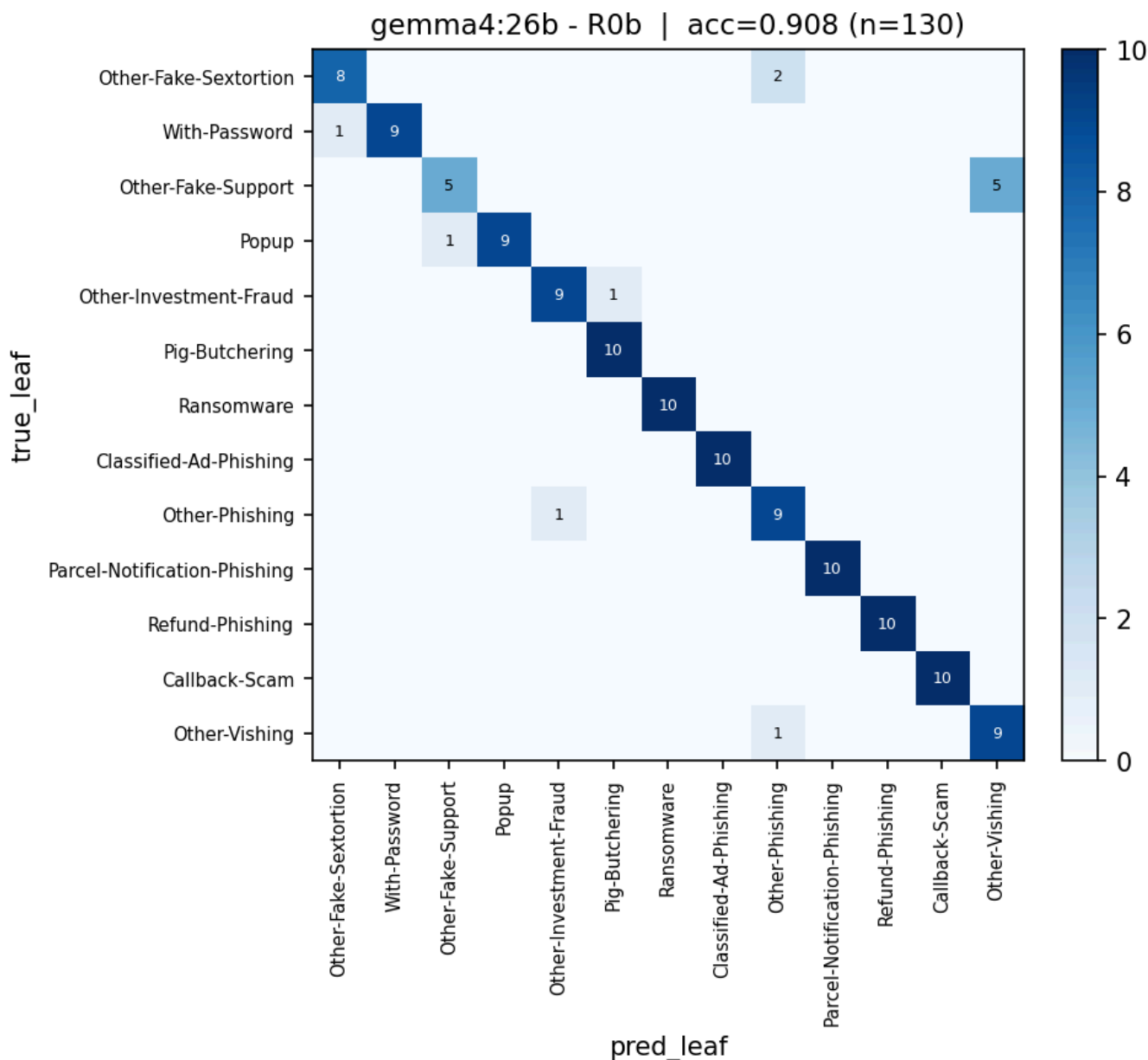


Figure 9. LLM 26B, R0b — confusion matrix (seed 0).

Adding examples (R8) does not help and costs one point (0.911). The e4b follows the same curve, lower: weak at R0a (0.463), it only becomes correct with the descriptions (0.822). Unlike the encoder, the LLM does not output probabilities: its errors cannot be sorted by confidence (§5.4).

5.4 Confidence analysis

For triage, you need a reliable confidence score. This is where the two families differ the most: the encoder outputs probabilities, the LLM a plain label. The hF1-AUC measures the quality of these confidences over all thresholds. At R8, it is 0.671 for the linear probe and 0.879 for SetFit (+0.208, 5/5 seeds): contrastive fine-tuning improves not only accuracy, it makes the confidences much more usable.

System (setting)	Coverage at $\leq 5\%$ error	Coverage at $\leq 10\%$ error
Sim. names (R0a)	4% $\pm$ 1%	5% $\pm$ 3%
Sim. descriptions (R0b)	31% $\pm$ 3%	37% $\pm$ 5%
Linear probe (R3)	14% $\pm$ 8%	29% $\pm$ 10%
SetFit (R3)	16% $\pm$ 17%	44% $\pm$ 15%
Linear probe (R8)	48% $\pm$ 5%	63% $\pm$ 7%
SetFit (R8)	58% $\pm$ 14%	72% $\pm$ 12%

Table 3. Share of the volume that can be automated under a given error threshold (5 seeds).

The effect is concrete. At a 5% tolerated error, the linear probe covers 48% of the volume, SetFit 58%; at 10%, 63% and 72%. SetFit does better on average, but its standard deviation is large (its coverage at 5% goes from 37% to 75% depending on the seed). The ranking SetFit > linear probe is solid on hF1-AUC (5/5); at this exact threshold, it stays indicative.

Confidence-threshold triage — risk vs coverage (6 systems)

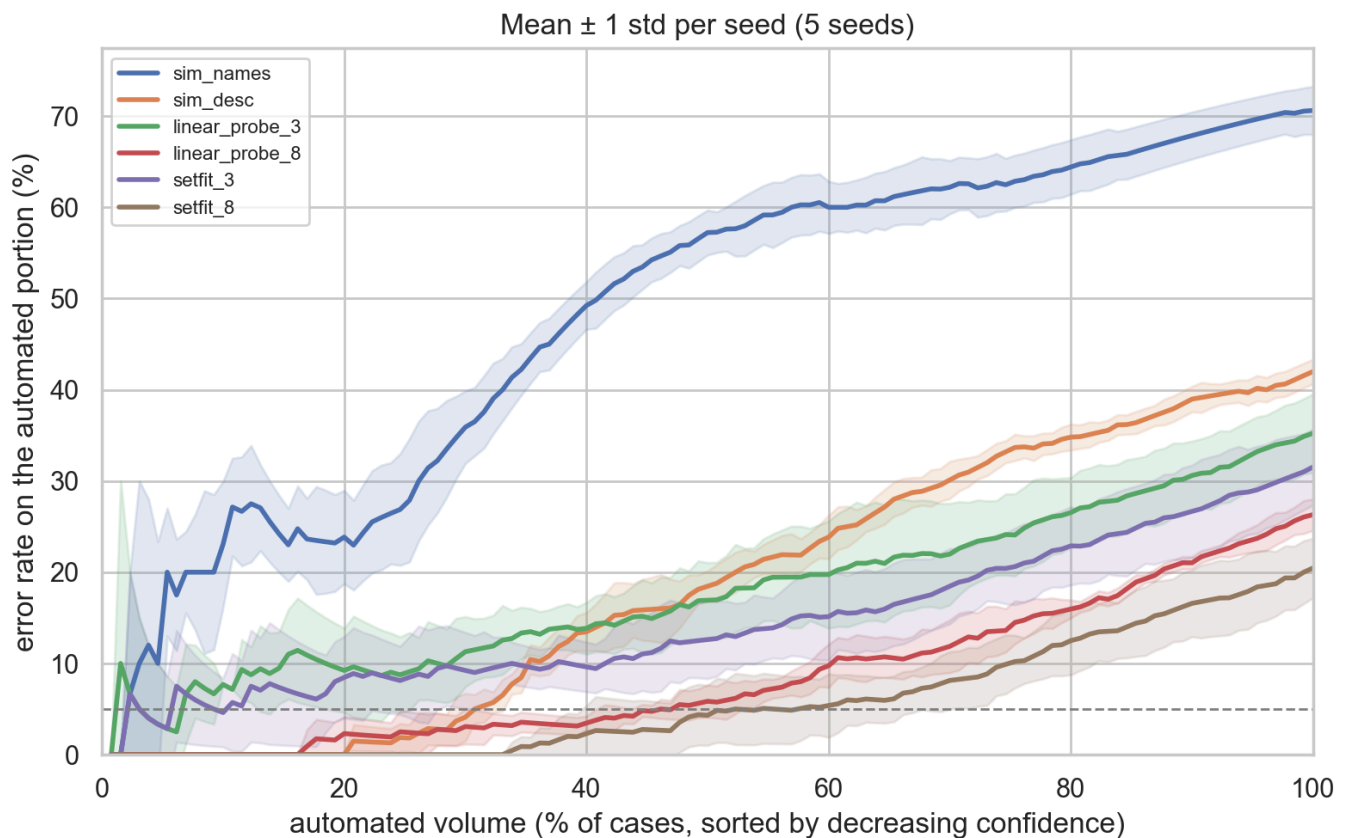


Figure 10. Error risk as a function of automated volume, by system (5 seeds).

5.5 Effect of contrastive fine-tuning on the latent space

Before fine-tuning, the families overlap; after, the categories group together and the families separate better, as a UMAP projection (McInnes et al., 2018) of the latent space shows (Figure 11). The silhouette at the family level goes from 0.053 to 0.141, with a similar tightening at level 2 (0.087 $\rightarrow$ 0.235). A few examples stay in the middle of another family; reading their text shows no label error — it is the encoder that places them there, not the label.

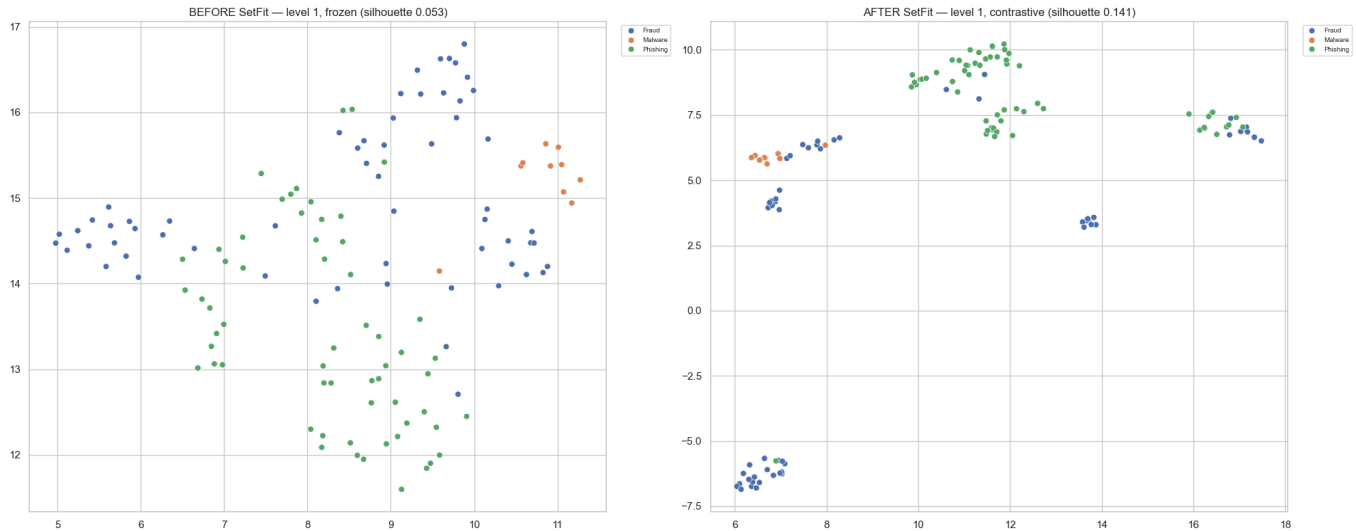


Figure 11. Latent space before/after contrastive fine-tuning, level 1 (UMAP, seed 0).

6. Discussion

6.1 Significance and uncertainty

Our conclusions are about the **sign** of the effects (5/5 or 0/5 differences), not their size (see the $n = 5$ floor, §4.4). The nested budgets also make the comparisons between settings paired: the $R0b \rightarrow R3$ drop for the LLMs is 0/5 for both models, so an effect, not noise. One exception: the e4b rise from $R3$ to $R8$ is positive on only 3 of 5 seeds (two seeds are an exact tie; $p = 0.125$), so not conclusive; only the 26B version of this rise is solid (5/5).

For the encoder, the variance between seeds is larger than for the LLM: on top of it come the split sampling and, for SetFit, the contrastive training (the LLM at temperature 0 has only the first source). But pairing absorbs this noise: each encoder comparison is 5/5 (linear probe vs SetFit at $R3$ as at $R8$, and at each budget step). The only place where the spread really weighs is coverage at a fixed threshold — hence the use of hF1-AUC, which sweeps all thresholds.

6.2 Economic reading

The cost of a system is not just its annotation budget: the way you inject the knowledge decides whether this budget is paid once or on every call. SetFit pays back its 104 examples in about 1 min of training per seed, then infers in about 5 ms/example. The 8-shot LLM re-pays a long

prompt (up to about 28,500 tokens) on every call. Let's be honest about how much this argument weighs: at NCSC scale and on the existing server, 0.6–1 s per example is about 10–16 h of compute per year — so throughput is not what decides. What does decide: how the prompt grows with the size of the taxonomy, and above all the lack of confidence scores on the LLM side.

For the first hour spent, the verdict is clear: the hour of descriptions beats the about 2 hours of the first 39 labels in both families, and on the LLM side in-context labels have a negative return. For the 26B, descriptions are both the best knowledge (0.922) and the cheapest to serve. So the recommendation depends on the deployment: if an LLM inference at about 1 s/example is acceptable and you do not need confidences, you stick with the descriptions.

If the deployment needs confidences to route to a human — the NCSC case — you go all the way to R8 with contrastive fine-tuning (SetFit): it is the system that maximises the volume that can be automated at low error (58% at $\leq 5\%$, 72% at $\leq 10\%$), with millisecond inference on CPU.

6.3 Limitations

- **English corpus only**, while the motivation is multilingual. The full grid still has to be checked on the other languages, German first, since it is 60% of the real cases.
- **One single branch** of the taxonomy (but the depth, 3 levels, is the maximum in production). The LLM prompt grows with the number of categories; the linear head does not care about it.
- **Texts pseudonymised upstream**; transfer to production noise still has to be checked.
- **The result "descriptions > few-shot" on the LLM side** holds for *these* descriptions and *these* models. Poorer or more detailed descriptions could move the crossing point.
- **Quality review by a single person.**
- **Possible contamination at pre-training**: some taxonomy labels or some modus operandi may be "known" to the model. One possible check: have it classify openly, without forcing the taxonomy, and watch which families the model falls back to.
- **The between-seed variance measures the re-splitting, not generalisation.** Between seeds, the same fixed pool of 234 examples is re-split; no seed brings new examples. So the reported standard deviation is a low (optimistic) bound on the real uncertainty.

7. Conclusion and outlook

On the branch we tested, the grid settles the allocation question, and it reads in three steps. The first franc should go to the definitions: describing the categories gives the best return in both families, and on the LLM side it is the only knowledge investment that still pays.

Annotations only pay off once they are put into the weights: added in the prompt, they make every call heavier for a result below the descriptions; in the encoder, contrastive fine-tuning makes them pay off. Finally, choosing a family is choosing capabilities more than raw accuracy: the LLM keeps the accuracy lead, but only the encoder gives the native confidence scores that human routing needs — the NCSC case.

Outlook, by priority: (1) multilingual (DE/FR/IT) — cross-lingual transfer with examples from the Module 6 dataset; (2) the full taxonomy; (3) using the confidences (calibration, routing threshold); (4) hierarchical prompting and LLM confidence (logprobs, self-consistency); (5) exploring SetFit hyperparameters.

Acknowledgements

I thank all the teachers and speakers of the CAS AML for the knowledge they shared; Dr. Mykhailo Vladymyrov for the feedback during the Module 6 presentation, which helped focus the project better; and my team at the NCSC, who helped collect the examples.

Generative AI tools. This work used AI tools: Claude Code (Anthropic) to help with code development, DeepL to translate the report into English, and ChatGPT (OpenAI) for literature search and implementation help. Extraction, curating the examples, and writing the descriptions were done by hand. The author remains responsible for all the code, the results, and the text, which he checked and validated.

References

- Alain, G., Bengio, Y. (2017). Understanding intermediate layers using linear classifier probes. *ICLR Workshop*. [PDF\(https://arxiv.org/pdf/1610.01644\)](https://arxiv.org/pdf/1610.01644)
- Brown, T., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*. arXiv:2005.14165. [PDF\(https://arxiv.org/pdf/2005.14165\)](https://arxiv.org/pdf/2005.14165)
- Bucher, M. J. J., Martini, M. (2024). Fine-Tuned 'Small' LLMs (Still) Significantly Outperform Zero-Shot Generative AI Models in Text Classification. arXiv:2406.08660. [PDF\(https://arxiv.org/pdf/2406.08660\)](https://arxiv.org/pdf/2406.08660)
- Chang, M.-W., Ratnikov, L., Roth, D., Srikumar, V. (2008). Importance of Semantic Representation: Dataless Classification. *AAAI 2008*. [PDF\(https://cogcomp.seas.upenn.edu/papers/CRRS08.pdf\)](https://cogcomp.seas.upenn.edu/papers/CRRS08.pdf)
- Edwards, A., Camacho-Collados, J. (2024). Language Models for Text Classification: Is In-Context Learning Enough? *LREC-COLING 2024*. arXiv:2403.17661. [PDF\(https://arxiv.org/pdf/2403.17661\)](https://arxiv.org/pdf/2403.17661)
- Google (2026). *Gemma 4 — 26B A4B model card*. Hugging Face / Google AI for Developers. [link\(https://huggingface.co/google/gemma-4-26B-A4B\)](https://huggingface.co/google/gemma-4-26B-A4B)

- Hugging Face. *sentence-transformers/paraphrase-multilingual-mpnet-base-v2* — *model card*. [link\(https://huggingface.co/sentence-transformers/paraphrase-multilingual-mpnet-base-v2\)](https://huggingface.co/sentence-transformers/paraphrase-multilingual-mpnet-base-v2)
- Kedzie, C., McKeown, K., Daumé III, H. (2018). Content Selection in Deep Learning Models of Summarization. *EMNLP*. arXiv:1810.12343. [PDF\(https://arxiv.org/pdf/1810.12343\)](https://arxiv.org/pdf/1810.12343)
- Kiritchenko, S., Matwin, S., Nock, R., Famili, A. F. (2006). Learning and Evaluation in the Presence of Class Hierarchies: Application to Text Categorization. *Advances in Artificial Intelligence (Canadian AI 2006)*, LNCS 4013, pp. 395–406. [DOI\(https://doi.org/10.1007/11766247_34\)](https://doi.org/10.1007/11766247_34)
- Kosmopoulos, A., et al. (2015). Evaluation measures for hierarchical classification. *Data Mining and Knowledge Discovery*, 29(3). [PDF\(https://arxiv.org/pdf/1306.6802\)](https://arxiv.org/pdf/1306.6802)
- McInnes, L., Healy, J., Melville, J. (2018). *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. arXiv:1802.03426. [PDF\(https://arxiv.org/pdf/1802.03426\)](https://arxiv.org/pdf/1802.03426)
- Mosbach, M., Andriushchenko, M., Klakow, D. (2021). On the Stability of Fine-tuning BERT. *ICLR*. [PDF\(https://arxiv.org/pdf/2006.04884\)](https://arxiv.org/pdf/2006.04884)
- Northcutt, C. G., Jiang, L., Chuang, I. L. (2021). *Confident Learning: Estimating Uncertainty in Dataset Labels*. *Journal of Artificial Intelligence Research*, 70. arXiv:1911.00068. [PDF\(https://arxiv.org/pdf/1911.00068\)](https://arxiv.org/pdf/1911.00068)
- Ollama. *gemma4 model library*. [link\(https://ollama.com/library/gemma4\)](https://ollama.com/library/gemma4)
- Plaud, R., Labeau, M., Saillenfest, A., Bonald, T. (2024). Revisiting Hierarchical Text Classification: Inference and Metrics. *CoNLL 2024*. arXiv:2410.01305. [PDF\(https://arxiv.org/pdf/2410.01305\)](https://arxiv.org/pdf/2410.01305)
- Reimers, N., Gurevych, I. (2019). Sentence-BERT. *EMNLP*. [PDF\(https://arxiv.org/pdf/1908.10084\)](https://arxiv.org/pdf/1908.10084)
- Silla, C. N., Freitas, A. A. (2011). A survey of hierarchical classification across different application domains. *DMKD*, 22(1–2). [DOI\(https://doi.org/10.1007/s10618-010-0175-9\)](https://doi.org/10.1007/s10618-010-0175-9)
- Sun, C., Qiu, X., Xu, Y., Huang, X. (2019). How to Fine-Tune BERT for Text Classification? *CCL*. arXiv:1905.05583. [PDF\(https://arxiv.org/pdf/1905.05583\)](https://arxiv.org/pdf/1905.05583)
- Tunstall, L., Reimers, N., et al. (2022). Efficient Few-Shot Learning Without Prompts (SetFit). arXiv:2209.11055. [PDF\(https://arxiv.org/pdf/2209.11055\)](https://arxiv.org/pdf/2209.11055)
- Yin, W., Hay, J., Roth, D. (2019). Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach. *EMNLP 2019*. arXiv:1909.00161. [PDF\(https://arxiv.org/pdf/1909.00161\)](https://arxiv.org/pdf/1909.00161)
- Zhang, T., Wu, F., Katiyar, A., Weinberger, K. Q., Artzi, Y. (2021). Revisiting Few-sample BERT Fine-tuning. *ICLR*. [PDF\(https://arxiv.org/pdf/2006.05987\)](https://arxiv.org/pdf/2006.05987)

More data

A lot of additional data and figures are available here: [notebooks/](#) and [artifacts/](#)